

CLAUDE CODE & CODEX CLI

Tapez moins, codez plus

Le guide complet des deux principaux agents de codage en ligne de commande : toutes les commandes, options et raccourcis, avec une comparaison détaillée pour choisir le bon outil au bon moment.

RÉDIGÉ ET VÉRIFIÉ • JUILLET 2026
ISMAËL LADJOHOUNLOU • [linkedin.com/in/ismaelladjohounlou](https://www.linkedin.com/in/ismaelladjohounlou)

Introduction

Claude Code (Anthropic) et Codex CLI (OpenAI) sont aujourd'hui les deux agents de codage en ligne de commande les plus utilisés. Les deux permettent de déléguer des tâches de développement complètes (lire du code, l'éditer, lancer des commandes shell, faire des recherches web) directement depuis le terminal, avec un haut degré d'autonomie.

Ce guide a deux objectifs :

1. Servir d'**aide-mémoire complet** pour Claude Code : toutes les commandes slash, options CLI, raccourcis clavier et mécanismes de configuration (CLAUDE.md, Skills, Hooks, MCP, Subagents, Plan Mode, Agent SDK).
2. Faire de même pour **Codex CLI**, puis proposer une **comparaison directe** entre les deux outils pour aider à choisir celui qui correspond le mieux à un usage donné.

Toutes les commandes et options listées ici sont reproduites exactement telles qu'elles doivent être tapées, y compris les tirets (-) qui séparent les mots dans les noms de commandes et de flags. Ne pas les modifier ni les traduire : `/code-review` et `--permission-mode` doivent être tapés lettre pour lettre.

Partie I : Claude Code

ANTHROPIC

Qu'est-ce que Claude Code ?

Claude Code est l'agent de codage en ligne de commande d'Anthropic. Il tourne dans le terminal (et existe aussi en extension VS Code, plugin JetBrains, application de bureau et application web sur claude.ai/code). Il lit et modifie des fichiers, exécute des commandes shell, cherche sur le web, et peut orchestrer des sous-agents pour des tâches complexes, le tout piloté par des instructions en langage naturel.

Le point d'entrée est la commande `claude` dans un terminal, lancée depuis la racine d'un projet.

Démarrage rapide

- `claude` : lance une session interactive dans le dossier courant
- `claude -p "corrige le bug dans auth.py"` : mode impression, une seule réponse puis sortie
- `claude --continue` : reprend la dernière conversation de ce dossier

Commandes slash

Les commandes slash se tapent directement dans la session interactive. Elles couvrent la gestion de session, la configuration, les outils intégrés et bien plus.

GESTION DE SESSION

COMMANDE	SYNTAXE	RÔLE
<code>/help</code>	<code>/help</code>	Affiche le menu d'aide et la liste des commandes
<code>/clear</code>	<code>/clear [nom]</code>	Démarre une nouvelle conversation, contexte vidé
<code>/exit</code>	<code>/exit</code> (ou Ctrl+D)	Quitte la session
<code>/rename</code>	<code>/rename [nom]</code>	Renomme la session en cours
<code>/resume</code>	<code>/resume [session]</code>	Reprend une conversation précédente par ID ou nom
<code>/rewind</code>	<code>/rewind</code>	Revient à un point de contrôle antérieur de la conversation
<code>/branch</code>	<code>/branch [nom]</code>	Crée une branche de conversation pour explorer une autre piste
<code>/background</code>	<code>/background [prompt]</code>	Détache la session en agent d'arrière-plan, libère le terminal

<code>/remote-control</code>	<code>/remote-control</code>	Rend la session pilotable à distance depuis claude.ai
------------------------------	------------------------------	---

CONTEXTE ET NAVIGATION

COMMANDE	SYNTAXE	RÔLE
<code>/cd</code>	<code>/cd <chemin></code>	Déplace la session vers un nouveau répertoire de travail
<code>/add-dir</code>	<code>/add-dir <chemin></code>	Ajoute des répertoires de travail supplémentaires
<code>/context</code>	<code>/context [all]</code>	Visualise l'utilisation du contexte sous forme de grille colorée
<code>/compact</code>	<code>/compact [instructions]</code>	Libère du contexte en résumant la conversation
<code>/memory</code>	<code>/memory</code>	Affiche et édite les fichiers CLAUDE.md et la mémoire automatique
<code>/recap</code>	<code>/recap</code>	Génère un résumé en une ligne de la session en cours
<code>/focus</code>	<code>/focus</code>	Bascule une vue épurée ne montrant que l'essentiel

MODÈLE ET CONFIGURATION

COMMANDE	SYNTAXE	RÔLE
<code>/model</code>	<code>/model [modèle]</code>	Change de modèle et l'enregistre par défaut
<code>/effort</code>	<code>/effort [niveau auto]</code>	Règle le niveau d'effort (low, medium, high, xhigh, max)
<code>/advisor</code>	<code>/advisor [modèle off]</code>	Active un second modèle conseiller
<code>/config</code>	<code>/config [clé=valeur ...]</code>	Ouvre les réglages ou fixe une préférence directement

REVUE ET QUALITÉ DU CODE

COMMANDE	SYNTAXE	RÔLE
<code>/code-review</code>	<code>/code-review [low medium high xhigh max ultra] [--fix] [--comment] [cible]</code>	Revoit le diff pour bugs et nettoyages
<code>/review</code>	<code>/review [PR]</code>	Revue rapide, lecture seule, d'une PR GitHub
<code>/security-review</code>	<code>/security-review</code>	Analyse les changements pour des failles de sécurité
<code>/simplify</code>	<code>/simplify [cible]</code>	Cherche des simplifications et les applique

<code>/diff</code>	<code>/diff</code>	Ouvre un visualiseur de diff interactif
--------------------	--------------------	---

TEST ET VÉRIFICATION

COMMANDE	SYNTAXE	RÔLE
<code>/run</code>	<code>/run</code>	Lance l'application pour valider les changements
<code>/verify</code>	<code>/verify</code>	Build + exécution pour confirmer que ça fonctionne
<code>/run-skill-generator</code>	<code>/run-skill-generator</code>	Apprend à <code>/run</code> et <code>/verify</code> comment lancer votre app
<code>/batch</code>	<code>/batch</code> <code><instruction></code>	Orchestre des changements massifs en parallèle sur le code

AUTOMATISATION ET PLANIFICATION

COMMANDE	SYNTAXE	RÔLE
<code>/loop</code>	<code>/loop [intervalle] [prompt]</code>	Répète un prompt tant que la session reste ouverte
<code>/schedule</code>	<code>/schedule [description]</code>	Crée, modifie, liste ou exécute des tâches planifiées
<code>/goal</code>	<code>/goal [condition clear]</code>	Fixe une condition à atteindre sur plusieurs tours

OUTILS ET INTÉGRATIONS

COMMANDE	SYNTAXE	RÔLE
<code>/mcp</code>	<code>/mcp [reconnect</code> <code><serveur> enable disable]</code>	Gère les connexions aux serveurs MCP
<code>/chrome</code>	<code>/chrome</code>	Configure l'extension Claude in Chrome
<code>/ide</code>	<code>/ide</code>	Gère les intégrations IDE et affiche leur statut
<code>/plugin</code>	<code>/plugin [sous-commande]</code>	Gère les plugins Claude Code
<code>/skills</code>	<code>/skills</code>	Liste les skills disponibles et gère leur visibilité
<code>/reload-plugins</code>	<code>/reload-plugins [--force]</code>	Recharge les plugins actifs
<code>/reload-skills</code>	<code>/reload-skills</code>	Rescane les dossiers de skills

PERMISSIONS ET ACCÈS

COMMANDE	SYNTAXE	RÔLE
<code>/permissions</code>	<code>/permissions</code>	Gère les règles de permission des outils (allow/deny/ask)
<code>/hooks</code>	<code>/hooks</code>	Affiche la configuration des hooks

DÉBOGAGE ET DIAGNOSTIC

COMMANDE	SYNTAXE	RÔLE
<code>/debug</code>	<code>/debug [description]</code>	Active les logs de débogage
<code>/doctor</code>	<code>/doctor</code>	Diagnostic et corrige les problèmes d'installation
<code>/heapdump</code>	<code>/heapdump</code>	Écrit un instantané mémoire pour diagnostic

COMPTE ET INFORMATIONS

COMMANDE	SYNTAXE	RÔLE
<code>/login</code>	<code>/login</code>	Connexion au compte Anthropic
<code>/logout</code>	<code>/logout</code>	Déconnexion
<code>/usage</code>	<code>/usage</code>	Suivi de l'usage API et des coûts
<code>/cost</code>	<code>/cost</code>	Alias de <code>/usage</code>
<code>/status</code>	<code>/status</code>	Ouvre les réglages sur l'onglet Statut
<code>/feedback</code>	<code>/feedback [rapport]</code>	Envoie un retour ou signale un bug

DIVERS

COMMANDE	SYNTAXE	RÔLE
<code>/plan</code>	<code>/plan [description]</code>	Entre directement en mode plan
<code>/agents</code>	<code>/agents</code>	Gère ou crée des configurations de sous-agents
<code>/copy</code>	<code>/copy [N]</code>	Copie la dernière réponse dans le presse-papiers
<code>/export</code>	<code>/export [fichier]</code>	Exporte la conversation en texte brut
<code>/keybindings</code>	<code>/keybindings</code>	Ouvre le fichier de raccourcis pour personnalisation
<code>/statusline</code>	<code>/statusline</code>	Configure la barre de statut
<code>/fast</code>	<code>/fast [on off]</code>	Active/désactive le mode rapide

<code>/sandbox</code>	<code>/sandbox</code>	Bascule le mode bac à sable pour l'exécution de commandes
<code>/install-github-app</code>	<code>/install-github-app</code>	Installe l'app GitHub de Claude sur un dépôt
<code>/install-slack-app</code>	<code>/install-slack-app</code>	Installe l'app Slack de Claude

Options de la commande `claude`

Ces options s'utilisent en ligne de commande, avant même de démarrer une session interactive.

SESSION ET MODÈLE

OPTION	ALIAS	RÔLE
<code>--print</code>	<code>-p</code>	Répond une fois sans mode interactif, puis quitte
<code>--continue</code>	<code>-c</code>	Charge la dernière conversation du dossier courant
<code>--resume</code>	<code>-r</code>	Reprend une session précise par ID ou nom
<code>--background</code>	<code>--bg</code>	Démarre en agent d'arrière-plan
<code>--model</code>		Choisit le modèle par alias ou nom complet
<code>--effort</code>		Niveau d'effort (low, medium, high, xhigh, max)
<code>--fallback-model</code>		Chaîne de modèles de secours

PERMISSIONS ET SÉCURITÉ

OPTION	RÔLE
<code>--permission-mode</code>	Mode de permission au démarrage (default, acceptEdits, plan, auto, dontAsk, bypassPermissions)
<code>--dangerously-skip-permissions</code>	Ignore toutes les demandes de permission
<code>--allowed-tools</code>	Pré-approuve des outils précis, ex. <code>"Bash(git *)"</code>
<code>--disallowed-tools</code>	Interdit des outils précis, ex. <code>"Bash(rm *)"</code>
<code>--tools</code>	Restreint la liste des outils intégrés disponibles

PROMPT SYSTÈME

OPTION	RÔLE
--------	------

<code>--system-prompt</code>	Remplace entièrement le prompt système
<code>--system-prompt-file</code>	Charge le prompt système depuis un fichier
<code>--append-system-prompt</code>	Ajoute du texte à la fin du prompt système par défaut
<code>--append-system-prompt-file</code>	Ajoute le contenu d'un fichier au prompt système

ENTRÉE / SORTIE

OPTION	RÔLE
<code>--output-format</code>	Format de sortie : text, json, stream-json
<code>--input-format</code>	Format d'entrée : text, stream-json
<code>--verbose</code>	Journalisation détaillée, tour par tour
<code>--json-schema</code>	Sortie JSON validée selon un schéma fourni

FICHIERS ET RÉPERTOIRES

OPTION	ALIAS	RÔLE
<code>--add-dir</code>		Ajoute des répertoires de travail supplémentaires
<code>--worktree</code>	<code>-w</code>	Démarre dans un worktree Git isolé
<code>--tmux</code>		Crée une session tmux pour le worktree

MCP ET PLUGINS

OPTION	RÔLE
<code>--mcp-config</code>	Charge des serveurs MCP depuis un fichier ou une chaîne JSON
<code>--strict-mcp-config</code>	N'utilise que les serveurs MCP fournis via <code>--mcp-config</code>
<code>--plugin-dir</code>	Charge un plugin depuis un dossier ou une archive <code>.zip</code>

PARAMÈTRES ET DIAGNOSTIC

OPTION	RÔLE
<code>--settings</code>	Chemin vers un fichier JSON de réglages, ou JSON en ligne
<code>--setting-sources</code>	Sources à charger, séparées par des virgules (user, project, local, managed)
<code>--safe-mode</code>	Démarre avec toutes les personnalisations désactivées
<code>--bare</code>	Mode minimal : pas de hooks, skills, plugins ni MCP auto-détectés

<code>--max-turns</code>	Limite le nombre de tours agentiques
<code>--max-budget-usd</code>	Plafond de dépense en dollars pour les appels API
<code>--debug</code>	Mode debug, avec filtrage optionnel par catégorie
<code>--version (-v)</code>	Affiche le numéro de version

Raccourcis clavier

RACCOURCI	ACTION
<code>Shift+Tab</code>	Change de mode de permission (default → acceptEdits → plan → auto → bypassPermissions)
<code>Ctrl+D</code>	Quitte la session (équivalent à <code>/exit</code>)
<code>Tab</code>	Complétion de commande
<code>↑</code>	Rappelle la commande précédente de l'historique
<code>Ctrl+C</code>	Interrompt l'opération en cours
<code>Ctrl+L</code>	Efface l'écran
<code>Ctrl+R</code>	Ouvre les sessions récentes / recherche dans l'historique
<code>Échap Échap</code>	Annule la saisie en cours
<code>Ctrl+U</code>	Efface la ligne courante
<code>Ctrl+W</code>	Supprime le mot précédent
<code>Ctrl+G</code>	Ouvre le plan proposé dans l'éditeur par défaut pour le modifier

Dans VS Code : `Ctrl+Shift+A` bascule la barre latérale Claude Code, `Ctrl+Shift+C` l'ouvre dans un nouvel onglet (Cmd sur macOS).

Hiérarchie des paramètres

Les réglages se combinent selon un ordre de priorité strict, du plus fort au plus faible :

1. **Managed** (imposé par l'organisation, non modifiable) : `/etc/claude-code/managed-settings.json` (Linux), registre Windows, ou plist macOS
2. **Arguments en ligne de commande**
3. **Local** : `.claude/settings.local.json` (ignoré par Git, propre à vous sur ce dépôt)
4. **Project** : `.claude/settings.json` (partagé avec l'équipe, versionné)
5. **User** : `~/.claude/settings.json` (vos préférences, tous projets)

Parmi les clés les plus utiles : `model` , `permissions.allow` / `permissions.deny` , `permissions.defaultMode` , `env` , `hooks` , `autoMemoryEnabled` , `disableBundledSkills` .

CLAUDE.md : la mémoire persistante

CLAUDE.md est un fichier markdown qui donne à Claude des instructions persistantes sur un projet, votre façon de travailler, ou toute une organisation. Claude le lit au démarrage de chaque session.

PORTÉE	EMPLACEMENT	CHARGEMENT
Organisation	selon plateforme (ex. <code>/etc/claude-code/CLAUDE.md</code>)	Systématique, non désactivable
Utilisateur	<code>~/.claude/CLAUDE.md</code>	Systématique
Projet	<code>./CLAUDE.md</code> ou <code>./.claude/CLAUDE.md</code>	Systématique
Local	<code>./CLAUDE.local.md</code> (ignoré par Git)	Systématique
Sous-dossiers	<code>./chemin/CLAUDE.md</code>	À la demande, quand Claude lit des fichiers de ce dossier

Les fichiers s'importent avec la syntaxe `@chemin/vers/fichier.md` (profondeur maximale : 4 niveaux). Des règles ciblées par chemin peuvent aussi être définies dans `.claude/rules/*.md` , avec un frontmatter `paths:` pour les limiter à certains fichiers.

À côté de CLAUDE.md existe la **mémoire automatique** : Claude note lui-même, sans y être invité, des correctifs et préférences observés (dans `~/.claude/projects/<projet>/memory/MEMORY.md`). Elle se désactive avec `"autoMemoryEnabled": false` .

Skills

Un **Skill** est un fichier `SKILL.md` qui étend les capacités de Claude. Contrairement à CLAUDE.md, son contenu n'est chargé en contexte que lorsqu'il est invoqué, via `/nom-du-skill` ou automatiquement quand Claude juge que c'est pertinent.

EMPLACEMENT	PORTÉE
<code>~/.claude/skills/<nom>/SKILL.md</code>	Personnel, tous vos projets
<code>.claude/skills/<nom>/SKILL.md</code>	Ce projet uniquement
Sous-dossier <code>skills/</code> d'un plugin	Là où le plugin est activé

Le frontmatter d'un Skill accepte notamment `name` , `description` , `argument-hint` , `allowed-tools` , `disallowed-tools` , `model` , `effort` , `context: fork` (exécution dans un sous-agent isolé), et `disable-model-invocation: true` pour n'autoriser que l'invocation manuelle.

Hooks

Les **Hooks** sont des commandes shell définies par l'utilisateur qui s'exécutent à des points précis du cycle de vie de Claude Code : un contrôle déterministe qui s'applique quoi que décide le modèle.

ÉVÉNEMENT	DÉCLENCHÉ	UTILE POUR
<code>SessionStart</code>	Au début de la session	Initialiser l'environnement, démarrer des serveurs
<code>Setup</code>	Au <code>/init</code> ou au démarrage	Installer des dépendances
<code>PreToolUse</code>	Avant l'exécution de tout outil	Valider, bloquer une action
<code>PostToolUse</code>	Après le succès d'un outil	Formater automatiquement, notifier
<code>PermissionRequest</code>	Quand une approbation est requise	Auto-approuver certaines actions
<code>PostCompaction</code>	Après un résumé du contexte	Réinjecter du contexte

Configuration dans `settings.json` , avec un objet `"if": "Nom(motif)"` et `"run": "commande"` .

MCP : Model Context Protocol

MCP est un standard ouvert pour connecter Claude Code à des outils, bases de données et API externes. Un serveur MCP fait le pont entre Claude et un système donné.

Ajout via `/mcp` , ou en CLI avec `--mcp-config ./mcp.json` . Fichier de configuration `.mcp.json` :

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "ghp_..." }
    }
  }
}
```

Les portées (local, project, user, managed) se fusionnent, les plus locales l'emportant sur les plus globales.

Subagents

Les **subagents** sont des assistants spécialisés qui traitent une tâche dans un contexte isolé, utile quand une sous-tâche produirait beaucoup de bruit (résultats de recherche, logs) sans intérêt pour la conversation principale. Chaque subagent a son propre contexte, prompt système, et accès aux outils.

Définis dans `.claude/agents/<nom>.md`, avec un frontmatter `name`, `description`, `model`, `tools`, `permission-mode`. Claude délègue automatiquement quand la description correspond à la tâche, ou peut être invoqué explicitement ("Utilise l'agent code-reviewer pour...").

Agents intégrés : **Explore** (recherche dans le code, lecture seule), **Plan** (planification), **general-purpose** (par défaut).

Plan Mode

Le **mode plan** est un mode de permission où Claude explore et propose des changements sans les exécuter. Il lit les fichiers et le code, puis rédige un plan structuré ; rien n'est édité ni exécuté avant votre validation.

Activation : `claude --permission-mode plan`, ou en session avec `/plan` ou `Shift+Tab`.

Après validation, plusieurs modes d'exécution sont proposés : approbation automatique complète, approbation automatique des seules éditions, ou revue manuelle de chaque étape.

Claude Agent SDK

Le **Claude Agent SDK** donne accès, sous forme de bibliothèque, au même moteur que Claude Code, pour construire ses propres agents en Python ou TypeScript, sur sa propre infrastructure.

LANGAGE	PACKAGE	INSTALLATION
Python	<code>claude-agent-sdk</code>	<code>pip install claude-agent-sdk</code> (Python 3.10+)
TypeScript	<code>@anthropic-ai/claude-agent-sdk</code>	<code>npm install @anthropic-ai/claude-agent-sdk</code>

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    async for message in query(
```

```
prompt="Trouve et corrige le bug dans auth.py",
options=ClaudeAgentOptions(allowed_tools=["Read", "Edit", "Bash"]),
):
    if hasattr(message, "result"):
        print(message.result)

asyncio.run(main())
```

Le SDK inclut par défaut les outils Read, Write, Edit, Bash, Glob, Grep, WebSearch, WebFetch, AskUserQuestion, et charge automatiquement `.claude/skills/`, `.claude/agents/` et `CLAUDE.md` depuis le répertoire de travail.

Partie II : Codex CLI

OPENAI

Qu'est-ce que Codex CLI ?

Codex CLI est l'agent de codage en ligne de commande d'OpenAI. Comme Claude Code, il tourne dans le terminal, lit et modifie du code, exécute des commandes shell, et peut fonctionner en mode autonome ou supervisé. Lancé sans sous-commande, `codex` ouvre une interface terminal interactive (TUI).

Démarrage rapide

`codex` : lance l'interface terminal interactive

`codex --sandbox workspace-write --ask-for-approval on-request` : travail local à faible friction

Commandes slash intégrées

COMMANDE	RÔLE
<code>/permissions</code>	Ajuste ce que Codex peut faire sans demander
<code>/approve</code>	Autorise à nouveau une action automatique récemment refusée
<code>/ide</code>	Intégration avec l'IDE
<code>/keymap</code>	Personnalise les raccourcis clavier
<code>/vim</code>	Active les raccourcis d'édition façon Vim
<code>/plugins</code>	Gère les plugins Codex
<code>/hooks</code>	Gère les hooks Codex
<code>/clear</code>	Vide la conversation en cours
<code>/rename</code>	Renomme la session
<code>/archive</code>	Archive une session sauvegardée
<code>/delete</code>	Supprime une session
<code>/compact</code>	Résume la conversation pour libérer du contexte
<code>/copy</code>	Copie la dernière sortie complétée de Codex
<code>/import</code>	Importe un contenu externe dans la session
<code>/feedback</code>	Envoie un retour à l'équipe Codex

<code>/init</code>	Initialise la configuration du projet
<code>/logout</code>	Déconnexion
<code>/mcp</code>	Gère les serveurs MCP connectés
<code>/mention</code>	Mentionne un fichier ou une ressource dans le prompt
<code>/model</code>	Change le modèle actif (ex. <code>gpt-5.4-mini</code> , <code>gpt-5.5</code>)
<code>/fast</code>	Active/désactive le mode rapide
<code>/plan</code>	Bascule en mode planification
<code>/app</code>	Actions liées à l'application
<code>/side</code>	Ouvre un panneau latéral
<code>/btw</code>	Ajoute une remarque annexe sans changer de sujet
<code>/raw</code>	Envoie une entrée brute, sans traitement
<code>/resume</code>	Reprend une session précédente
<code>/new</code>	Démarre une nouvelle session
<code>/quit</code>	Quitte la CLI (comme <code>/exit</code>)
<code>/review</code>	Demande à Codex de revoir l'arbre de travail local
<code>/status</code>	Affiche la configuration de session et l'usage de tokens
<code>/theme</code>	Choisit un thème de coloration syntaxique du terminal
<code>/pets</code> (ou <code>/pet</code>)	Choisit ou masque un compagnon animé dans le terminal

`/quit` et `/exit` quittent tous deux la CLI : à utiliser seulement après avoir sauvegardé ou committé son travail. `/status` confirme le modèle actif, la politique d'approbation, les répertoires accessibles en écriture et la capacité de contexte restante.

Options globales

OPTION	ALIAS	VALEURS / RÔLE
<code>--model</code>	<code>-m</code>	Remplace le modèle défini en configuration (ex. <code>gpt-5.4</code>)
<code>--image</code>	<code>-i</code>	Joint une ou plusieurs images au prompt initial (chemins séparés par des virgules)
<code>--local-provider</code>		<code>lmstudio</code> <code>ollama</code> : fournisseur local utilisé avec <code>--oss</code>
<code>--no-alt-screen</code>		Désactive le mode plein écran alternatif de la TUI

<code>--oss</code>		Utilise un modèle open source local
<code>--profile</code>	<code>-p</code>	Superpose <code>\$CODEX_HOME/profile-nom.config.toml</code> à la config de base
<code>--remote</code>		Connexion à un serveur d'application distant via WebSocket ou socket Unix (<code>ws://</code> , <code>wss://</code> , <code>unix://</code>)
<code>--remote-auth-token-env</code>		Variable d'environnement contenant un jeton d'authentification pour <code>--remote</code>
<code>--sandbox</code>	<code>-s</code>	<code>read-only</code> <code>workspace-write</code> <code>danger-full-access</code> : politique du bac à sable pour les commandes shell générées par le modèle

Sous-commandes

COMMANDE	STATUT	RÔLE
<code>codex app-server</code>	Expérimental	Lance le serveur d'application Codex (stdio, WebSocket, ou socket Unix)
<code>codex apply</code> (alias <code>codex a</code>)	Stable	Applique le dernier diff généré par une tâche cloud Codex
<code>codex archive</code>	Stable	Archive une session interactive sauvegardée par ID ou nom
<code>codex cloud</code> (alias <code>codex cloud-tasks</code>)	Expérimental	Parcourt ou exécute des tâches cloud Codex depuis le terminal
<code>codex completion</code>	Stable	Génère des scripts de complétion shell (Bash, Zsh, Fish, PowerShell)
<code>codex resume</code>	Stable	Reprend une session sauvegardée
<code>codex fork</code>	Stable	Duplique une session existante dans une nouvelle
<code>codex delete</code>	Stable	Supprime une session sauvegardée
<code>codex unarchive</code>	Stable	Désarchive une session

Pour connecter la TUI à un serveur d'application déjà lancé : `codex app-server --listen ws://IP:PORT`, puis se connecter avec `--remote ws://host:port`. En local, utiliser `--remote unix://` pour le socket par défaut.

Raccourcis interactifs

RACCOURCI	ACTION
@	Cherche un fichier dans l'espace de travail et ajoute son chemin au prompt

↑ / ↓	Restaure l'historique des brouillons
Ctrl+R	Cherche dans l'historique des prompts, Entrée pour valider, Échap pour annuler
Ctrl+O (ou /copy)	Copie la dernière sortie complétée de Codex
! en début de ligne	Exécute une commande shell locale sous les réglages de bac à sable actuels
Tab (pendant l'exécution)	Met en file un prompt, une commande slash ou une commande shell pour le tour suivant
Entrée (pendant l'exécution)	Injecte de nouvelles instructions dans le tour en cours
Échap Échap (composeur vide)	Édite le dernier message utilisateur et crée une branche depuis ce point
Ctrl+C (ou /exit)	Ferme la session

Configuration

Codex utilise deux mécanismes principaux :

- **AGENTS.md** : l'équivalent de CLAUDE.md, instructions persistantes pour le projet, à la racine du dépôt.
- **~/.codex/config.toml** : configuration globale (modèle par défaut, sandbox, fournisseurs, profils).

Un système de **profils** permet de superposer plusieurs configurations (**--profile nom** charge **\$CODEX_HOME/profile-nom.config.toml** par-dessus la config de base), utile pour basculer rapidement entre des réglages "strict" et "permissif" par exemple.

Partie III : Comparaison

Claude Code vs Codex CLI

ASPECT	CLAUDE CODE	CODEX CLI
Éditeur	Anthropic	OpenAI
Modèles	Famille Claude (Sonnet, Opus, Haiku)	Famille GPT (ex. gpt-5.4, gpt-5.5)
Fichier de mémoire projet	CLAUDE.md	AGENTS.md
Config globale	~/.claude/settings.json (JSON)	~/.codex/config.toml (TOML)
Extensibilité	Skills (SKILL.md), Subagents (.claude/agents/), Plugins	Plugins Codex, profils de config
Intégration outils externes	MCP (.mcp.json , /mcp)	MCP (/mcp)
Automatisations shell	Hooks (SessionStart, PreToolUse, PostToolUse, etc.)	Hooks Codex (/hooks)
Mode planification	Plan Mode (--permission-mode plan , Shift+Tab)	/plan
Bac à sable	--permission-mode , --allowed-tools / --disallowed-tools	--sandbox (read-only / workspace-write / danger-full-access)
Session distante	Remote Control depuis claude.ai, --teleport	--remote (WebSocket / socket Unix) vers codex app-server
SDK programmable	Claude Agent SDK (Python / TypeScript)	Codex fournit un app-server exposé en API locale
Modèles locaux / open source	Non natif	--oss , --local-provider (LM Studio, Ollama)
Mascotte terminal	aucune	/pets (compagnon animé dans le terminal)

Forces respectives

CLAUDE CODE

Écosystème d'extensibilité le plus riche à ce jour : Skills invocables (par vous ou automatiquement par le modèle), Subagents avec contexte et permissions isolés, Hooks à sept points du cycle de vie, un SDK complet pour construire ses propres agents en production. La mémoire automatique (au-delà de CLAUDE.md) permet à l'outil d'apprendre en continu vos préférences sans intervention.

CODEX CLI

Interface terminal légère et rapide, avec un système de profils de configuration TOML pratique pour basculer entre contextes (ex. profil "prod" restrictif vs profil "exploration" permissif). Le support natif des modèles locaux/open source (`--OSS` , LM Studio, Ollama) en fait un choix pertinent pour du travail hors-ligne ou sur infrastructure privée.

Quel outil pour quel usage ?

Choisir Claude Code si : vous voulez construire des workflows complexes et réutilisables (Skills, Subagents, Hooks), automatiser des tâches programmatiquement via le SDK, ou bénéficier d'un écosystème de plugins et de MCP mature.

Choisir Codex CLI si : vous privilégiez une CLI légère bien intégrée à l'écosystème OpenAI, avez besoin de faire tourner des modèles en local pour des raisons de coût ou de confidentialité, ou préférez une configuration par profils TOML.

En pratique, les deux outils sont suffisamment proches dans leur philosophie (agent autonome, permissions graduées, mémoire persistante par fichier markdown à la racine du projet) pour être utilisés en parallèle selon la tâche : rien n'empêche d'avoir `CLAUDE.md` et `AGENTS.md` dans le même dépôt.

Aide-mémoire rapide

CLAUDE CODE : L'ESSENTIEL

```
claude                                # session interactive
claude -p "question"                 # réponse unique, sans interaction
claude --continue                     # reprend la dernière session
claude --resume nom-session           # reprend une session précise
claude --permission-mode plan         # démarre en mode plan
claude --model claude-opus-4-8        # choix du modèle
claude -w feature-x                   # worktree Git isolé
/compact                             # libère du contexte
/model                               # change de modèle en session
Shift+Tab                             # change de mode de permission
Ctrl+R                               # historique de sessions
```

CODEX CLI : L'ESSENTIEL

```
codex                                # interface interactive
codex --sandbox workspace-write \
    --ask-for-approval on-request     # travail local low-friction
codex -m gpt-5.5                      # choix du modèle
codex --oss --local-provider ollama   # modèle local via Ollama
codex resume                          # reprend une session
codex apply                           # applique le diff d'une tâche
cloud
/model                               # change de modèle en session
/status                              # config + usage de tokens
@                                     # référence un fichier au
prompt
Ctrl+R                               # historique des prompts
```